

Plano de Ação — Painel de Controle Universal Multi-Harness / Multi-LLM

Status: planejado (implementação incremental em `feature/painel-controle-multi-harness`)

Data: 2026-08-13 **Escopo:** frontend + backend local para operar a Fábrica de Materiais de Comunicação sem depender de uma sessão interativa de Claude Code, com provedor de LLM/harness escolhido pelo usuário, credenciais próprias do usuário, e todos os artefatos gerados dentro de uma pasta que o próprio usuário define — sem banco de dados de conteúdo.

1. Contexto

Hoje a Fábrica roda 100% dentro de uma sessão de agente (Claude Code, opencode, etc.) guiada por skills e `SPEC_COMANDOS.md`. Não existe frontend, não existe processo de longa duração, e a única forma de operar é via chat/slash-command dentro do harness.

O repositório já resolveu parcialmente o problema de “universalidade” no nível de **descoberta de comando/skill** (ver `docs/08-universalidade-harnesses.md`, REGRA 10 do `AGENTS.md`): existe uma arquitetura de 3 camadas — canônico único (`SPEC_COMANDOS.md/AGENTS.md/skills`) + adaptadores finos por harness (`.claude/commands/`, `.opencode/commands/`) + rules finas por harness (`CLAUDE.md`, `GEMINI.md`, `CODEBUDDY.md`, `QODER.md`). Isso é a base que este plano estende: em vez de um humano abrir um harness manualmente, um **backend local** faz isso por ele, de forma headless, escolhendo qual harness/credencial/modelo usar.

2. Objetivo

Dar “cara” à Fábrica (frontend + fluxo orientado a formulário) sem: - reescrever nenhuma skill, SPEC ou script existente; - travar o produto a um único provedor de LLM (Anthropic/OpenAI/Google/etc.); - travar o produto a um único harness (Claude Code); - introduzir banco de dados para os artefatos de conteúdo — eles continuam sendo arquivos, dentro de uma pasta (workspace) que o usuário escolhe antes de iniciar.

3. Decisão de arquitetura central

“Escolher provedor/LLM” mapeia para “escolher harness headless configurado com as credenciais/modelo desse harness”, não para “chamar a API crua do provedor”.

Motivo: a Fábrica é consumida hoje por um **agente com ferramentas** (Bash, leitura/ escrita de arquivo, Playwright, Pandoc/Typst, fan-out de subagentes) — não por um prompt único enviado a um endpoint de chat. Reimplementar isso contra APIs cruas de LLM significaria reconstruir um harness do zero (SDK de agente próprio), o que viola diretamente o requisito de “reaproveitar 100% sem reescrever”.

O que já cobre “múltiplos provedores” sem nenhuma reescrita: - **opencode**: já é nativamente multi-provedor/BYOK (Anthropic, OpenAI, Google, local, etc.), e o repo já tem `opencode.jsonc` configurado. - **Claude Code headless** (`claude -p`): cobre Anthropic (direto, Bedrock ou Vertex). - Outros harnesses (Gemini CLI, Cursor CLI, etc.) entram depois, um adaptador fino por vez, seguindo a mesma arquitetura de 3 camadas já validada pelo repo.

4. Componentes do sistema

Componente	Responsabilidade	Onde vive
Workspace	Pasta raiz escolhida pelo usuário; todos os projetos/artefatos (config_projeto.json, brief_criativo.json, output/<slug>/...) são gravados só ali	dentro da pasta do usuário (caminho arbitrário, fora do repo)
Cofre de credenciais	Guarda API keys/tokens por harness, criptografado, nunca dentro do workspace	~/fabrica-painel/vault.enc (fora do workspace)
Índice de execuções	Bookkeeping da aplicação (job id, harness, modelo, status, timestamps, log) — não é o conteúdo gerado, é só histórico de execução	SQLite em ~/fabrica-painel/painel.db (fora do workspace)
Registry de adaptadores de harness	Um adaptador fino por harness: monta comando headless, variáveis de ambiente (credencial/modelo) e cwd = pasta do projeto	novo código, ex.: painel/harness_adapters/*.py
Backend local (FastAPI)	Expõe API REST: registrar workspace, salvar credencial, criar projeto, disparar produção, consultar status	novo código, painel/
Frontend mínimo	Wizard (pasta + harness + credencial + modelo) e dashboard (projetos, progresso, download)	HTML/JS estático servido pelo backend

5. Fluxo de uso

1. Usuário abre o painel local (uvicorn painel.main:app → localhost:8787).
2. Wizard: escolhe a pasta de trabalho (workspace), escolhe harness (Claude Code / opencode / outro), informa credencial e modelo.
3. Cria um novo projeto (formulário substitui a entrevista conversacional do /esbocar — como é headless, é “one-shot”: todas as respostas vão de uma vez, sem ida-e-volta de chat). O backend grava config_projeto.json dentro do workspace.

4. Dispara produção → backend spawna o harness escolhido, headless, com cwd = pasta do projeto, rodando o comando canônico (/produzir-comunicacao-completa <slug>), usando a credencial/modelo escolhidos.
5. Dashboard consulta status lendo manifesto_materiais.json/_pool_estado.json direto da pasta do projeto (sem duplicar em banco) + status de processo do índice SQLite (rodando/concluído/erro).
6. Artefatos ficam 100% na pasta do usuário; download é servir o arquivo direto dali.

6. Riscos e limitações conhecidas

- **Não é um serviço cloud multiusuário** — é uma aplicação local (backend na máquina do usuário), porque precisa spawnar processo e escrever em pasta arbitrária do disco. Um browser puro não consegue fazer isso.
- **Headless ≠ entrevista conversacional real**: /esbocar foi desenhado para diálogo humano (REGRA 3). No fluxo headless, o formulário precisa antecipar tudo — perde-se a adaptação dinâmica de perguntas de acompanhamento que a conversa permite.
- **Cada harness tem sintaxe própria de invocação headless e de credencial** — o adaptador precisa ser validado contra a CLI real instalada; nesta primeira entrega, os adaptadores de harness real (claude-code, opencode) são validados só por construção de comando (testes mockados), não por chamada real a LLM (evitar custo e risco de recursão de agente dentro desta própria sessão). Um adaptador echo/dry-run é usado para provar a canalização (subprocess → status → log) de ponta a ponta com processo real, sem depender de nenhuma credencial.
- **Cofre de credenciais local com criptografia simples (Fernet + chave local)** é adequado para uso pessoal single-user, não é um cofre de nível empresarial (sem rotação de chave, sem HSM).

7. Plano de implementação (passo a passo, com validação antes de avançar)

1. Scaffold painel/ + banco de índice SQLite (fora do workspace) — testes unitários.
2. Módulo de workspace (registrar/validar pasta escolhida pelo usuário) — testes.
3. Cofre de credenciais criptografado — testes.
4. Registry de adaptadores de harness (base + echo + claude-code + opencode) — testes mockados para os adaptadores reais, teste real (subprocess de verdade) para o adaptador echo.
5. Job runner (spawna subprocess real, atualiza status/log no índice) — teste real com adaptador echo.
6. API FastAPI ligando os módulos anteriores — testes via TestClient.
7. Frontend mínimo (wizard + dashboard) servido pelo backend.
8. Smoke test end-to-end real: subir o servidor, exercitar o fluxo completo via HTTP (workspace temporário + adaptador echo), confirmar arquivos na pasta e job no índice.
9. README do painel (como rodar, como configurar harness real, o que foi validado de verdade vs. o que precisa de credencial real do usuário para validar).
10. Relatório final consolidando o que foi entregue e testado.

8. Reversibilidade

Toda a implementação vive em `feature/painel-controle-multi-harness`, em uma pasta nova (`painel/`) que não modifica nenhum arquivo existente do pipeline (skills, scripts, SPECs). Se a decisão for não seguir com o projeto, basta não fazer merge da branch — `main` permanece intocado.